Laboratório 11

CoAP e HTTP: Mensagens, Descoberta e Retransmissão

Tecnologias de Comunicação para a Internet das Coisas | Aula 11

Objetivo e enquadramento

A Aula 11 compara o HTTP, o CoAP e o XMPP. Este laboratório torna observáveis as diferenças mais importantes entre o HTTP e o CoAP: o tamanho das mensagens, a descoberta de recursos e a fiabilidade sobre UDP. Para isso, usa um servidor e um cliente CoAP minimalistas, escritos em Python, que trocam mensagens reais em `localhost`.

O trabalho não requer hardware nem máquina virtual, e corre em qualquer computador com Python 3. Destina-se a ser realizado durante a aula, ao ritmo de cada aluno, e **não constitui trabalho a entregar**.

Nota

O programa implementa apenas o necessário: cabeçalho base, token, opção Uri-Path e carga útil, conforme a RFC 7252. Não substitui uma biblioteca completa, como o `aiocoap`.

Resultados de aprendizagem

- Interpretar os octetos de uma mensagem CoAP e identificar os seus campos.
- Comparar o tamanho de um pedido e de uma resposta em CoAP e em HTTP.
- Descobrir os recursos de um servidor através de `.well-known/core`.
- Observar a retransmissão de uma mensagem confirmável com recuo exponencial.

Requisitos

- Computador com Windows, macOS ou Linux e Python 3.9 ou superior.
- Duração estimada: entre 40 e 60 minutos.

Como utilizar este guião

Cada passo apresenta, por esta ordem, o objetivo, o enquadramento, o fundamento teórico, os comandos a executar e o resultado esperado. Recomenda-se ler o passo completo antes de executar o primeiro comando. As caixas têm sempre o mesmo significado:

Comando a executar

```
Comando a executar no terminal.
```

Resultado esperado (os valores podem variar)

```
Resultado que deve ser apresentado.
```

Analisar

Momento de pausa para interpretar o resultado obtido, isoladamente ou em discussão com os colegas.

Erro possível e resolução

Erro que pode ocorrer e respetiva resolução.

Percurso



Servidor e cliente CoAP escritos em Python, a correr em localhost

Como ler a figura

Os cinco passos leem-se da esquerda para a direita. O primeiro prepara o programa. Os passos 2 e 3, a verde, comparam mensagens e descobrem recursos. O passo 4 observa a retransmissão com perdas, e o passo 5 encerra o trabalho.

Passo 1 — Preparação do programa

Objetivo. Guardar o programa do laboratório.

Fundamento teórico

O programa lança um servidor CoAP numa thread, sobre um socket UDP em `127.0.0.1`, e um cliente que constrói mensagens em bruto, octeto a octeto. Cada comando, `sizes`, `discover` e `retransmit`, corresponde a um passo do guião. Sem argumento, executa os três.

Criar o ficheiro `coap_lab.py` com o conteúdo seguinte. O ficheiro está também na pasta `codigo` do material da aula.

Conteúdo do ficheiro `coap_lab.py`

```
"""CoAP vs HTTP lab (Lecture 11): message sizes, resource discovery and
    confirmable retransmission.

Run:  python3 coap_lab.py [sizes|discover|retransmit]
Only the Python standard library is used; everything runs on localhost.
"""
import socket
import struct
import sys
import threading
import time

# ---------------------------------------------------------------- CoAP codec (RFC
    7252, minimal)
CON, NON, ACK, RST = 0, 1, 2, 3
GET, CONTENT, NOT_FOUND = 1, 69, 132             # codes 0.01, 2.05, 4.04
RESOURCES = {"temp": b"23.5", ".well-known/core": b'</temp>;ct=0;rt="temperature
    "'}


def encode_options(path):
    """Uri-Path options (number 11), one per path segment."""
    out, prev = b"", 0
    for seg in path.split("/"):
        data = seg.encode()
        out += bytes([((11 - prev) << 4) | len(data)]) + data     # segments are
    short here
        prev = 11
    return out
```

```
def build(mtype, code, mid, token=b"", path="", payload=b""):
    head = struct.pack("!BBH", (1 << 6) | (mtype << 4) | len(token), code, mid)
    msg = head + token + (encode_options(path) if path else b"")
    return msg + (b"\xff" + payload if payload else b"")


def parse(data):
    ver_t_tkl, code, mid = struct.unpack("!BBH", data[:4])
    tkl = ver_t_tkl & 0x0F
    i, path, prev = 4 + tkl, [], 0
    while i < len(data) and data[i] != 0xFF:
        delta, ln = data[i] >> 4, data[i] & 0x0F
        prev += delta
        if prev == 11:
            path.append(data[i + 1:i + 1 + ln].decode())
        i += 1 + ln
    payload = data[i + 1:] if i < len(data) and data[i] == 0xFF else b""
    return dict(type=(ver_t_tkl >> 4) & 3, code=code, mid=mid, token=data[4:4 +
    tkl], path="/".join(path), payload=payload)


# ------------------------------------------------------------ server and client
def server(sock, drop_first=0):
    seen = 0
    while True:
        try:
            data, addr = sock.recvfrom(1500)
        except OSError:
            return
        seen += 1
        if seen <= drop_first:
            continue                                    # simulate a lost request
        req = parse(data)
        body = RESOURCES.get(req["path"])
        code = CONTENT if body is not None else NOT_FOUND
        sock.sendto(build(ACK, code, req["mid"], req["token"], payload=body or b
    ""), addr)


def start_server(drop_first=0):
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    s.bind(("127.0.0.1", 0))
    threading.Thread(target=server, args=(s, drop_first), daemon=True).start()
    return s, s.getsockname()


def get(addr, path, mid=1, timeout=0.5, tries=4):
```

```
    msg = build(CON, GET, mid, token=b"\x42", path=path)
    c = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    for attempt in range(tries):
        c.settimeout(timeout * (2 ** attempt))        # exponential back-off
        c.sendto(msg, addr)
        t0 = time.time()
        try:
            data, _ = c.recvfrom(1500)
            return msg, data, attempt, time.time() - t0
        except socket.timeout:
            print(f"  attempt {attempt + 1}: no ACK within {timeout * 2 ** attempt
    :.1f} s, retransmitting")
    return msg, None, tries, None


# ---------------------------------------------------------------- commands
def sizes():
    s, addr = start_server()
    msg, resp, _, _ = get(addr, "temp")
    http_req = (b"GET /temp HTTP/1.1\r\nHost: 192.168.1.50:8080\r\nUser-Agent:
    sensor/1.0\r\n"
                b"Accept: text/plain\r\nConnection: close\r\n\r\n")
    http_resp = (b"HTTP/1.1 200 OK\r\nContent-Type: text/plain\r\nContent-Length:
    4\r\n"
                 b"Server: sensor/1.0\r\nConnection: close\r\n\r\n23.5")
    print(f"CoAP request : {len(msg):3d} octets  {msg.hex(' ')}")
    print(f"CoAP response: {len(resp):3d} octets  {resp.hex(' ')}")
    print(f"HTTP request : {len(http_req):3d} octets (plus TCP: 3-way handshake
    and close)")
    print(f"HTTP response: {len(http_resp):3d} octets")
    p = parse(resp)
    print(f"parsed response: type={p['type']} code={p['code']} (2.05 Content)
    payload={p['payload']!r}")
    print(f"payload is {len(p['payload'])} octets of {len(resp)} in the CoAP
    response")
    s.close()


def discover():
    s, addr = start_server()
    for path in (".well-known/core", "temp", "humidity"):
        _, resp, _, _ = get(addr, path)
        p = parse(resp)
        cls = "2.05 Content" if p["code"] == CONTENT else "4.04 Not Found"
        print(f"GET /{path:<18} -> {cls}  {p['payload'].decode() or '-'}")
    s.close()
```

```
def retransmit():
    s, addr = start_server(drop_first=2)
    print("server drops the first two requests")
    msg, resp, attempt, dt = get(addr, "temp", timeout=0.5)
    print(f"answered after {attempt} retransmission(s); payload {parse(resp)['
    payload']!r}")
    s.close()


CMDS = {"sizes": sizes, "discover": discover, "retransmit": retransmit}

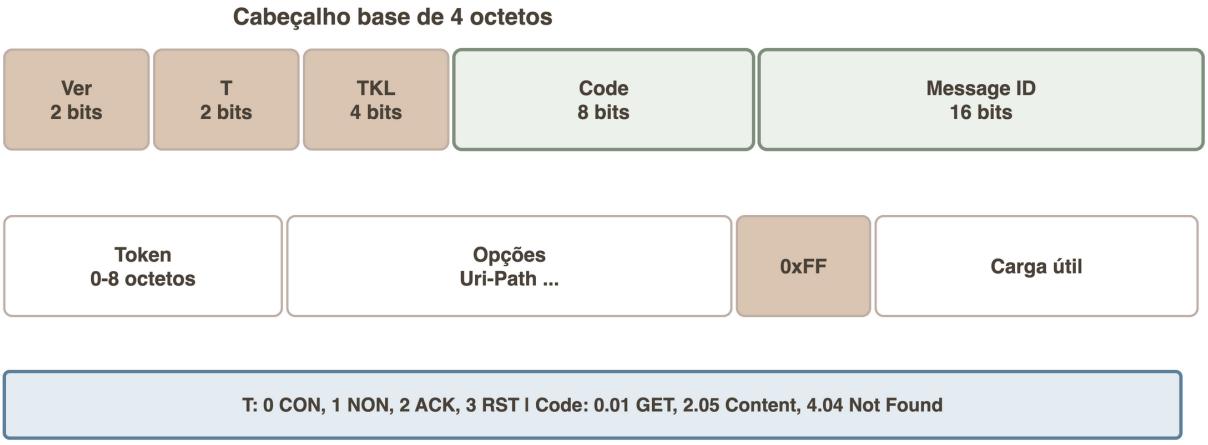
if __name__ == "__main__":
    which = sys.argv[1] if len(sys.argv) > 1 else None
    for name, fn in CMDS.items():
        if which in (None, name):
            print(f"== {name}")
            fn()
```

Erro possível e resolução

python3: command not found em Windows: usar `python coap_lab.py sizes`.

Passo 2 — Tamanho das mensagens

Objetivo. Interpretar os octetos de um pedido e de uma resposta CoAP e compará-los com HTTP.



Como ler a figura

A figura é a do diapositivo: o cabeçalho base de 4 octetos, seguido do token, das opções, do marcador `0xFF` e da carga útil. Cada campo do resultado do programa corresponde a uma destas caixas.

Fundamento teórico

O primeiro octeto `41` em hexadecimal significa versão 1, tipo 0 (confirmável) e comprimento de token 1. O segundo, `01`, é o código GET. Os dois seguintes, `00 01`, são o identificador da mensagem. Segue-se o token `42` e a opção Uri-Path, `b4 74 65 6d 70`, em que `b` é o delta 11 e `4` o comprimento da cadeia `temp`. Na resposta, `61` indica tipo ACK e `45` o código 2.05 (Content), seguido do marcador `ff` e da carga `23.5`.

A analisar neste passo: quantos octetos são cabeçalho e quantos são dados.

Comando a executar

```
python3 coap_lab.py sizes
```

Resultado esperado (os valores podem variar)

```
CoAP request :  10 octets  41 01 00 01 42 b4 74 65 6d 70
CoAP response:  10 octets  61 45 00 01 42 ff 32 33 2e 35
```

```
HTTP request : 110 octets (plus TCP: 3-way handshake and close)
HTTP response: 107 octets
parsed response: type=2 code=69 (2.05 Content) payload=b'23.5'
payload is 4 octets of 10 in the CoAP response
```

Analisar

Quantos octetos de dados úteis tem a resposta CoAP, e que fração do total representam? O pedido HTTP mostrado é mínimo. Que octetos acrescentariam um `Cookie` ou um `User-Agent` típico de um navegador? E o que custou, em mensagens, abrir a ligação TCP?

Passo 3 — Descoberta de recursos

Objetivo. Perguntar a um servidor que recursos oferece e observar um erro 4.04.

Fundamento teórico

O CoAP define um recurso especial, `.well-known/core`, que devolve a lista de recursos do servidor no formato de ligação do CoRE (RFC 6690). Um cliente pode assim descobrir o que existe sem conhecimento prévio. Um recurso inexistente devolve o código 4.04, equivalente ao 404 do HTTP.

Comando a executar

```
python3 coap_lab.py discover
```

Resultado esperado (os valores podem variar)

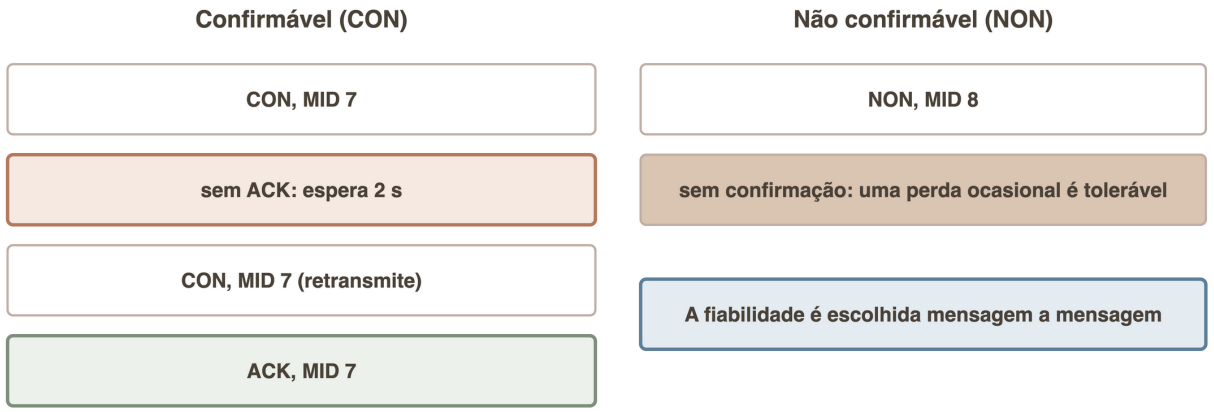
```
GET /.well-known/core   -> 2.05 Content  </temp>;ct=0;rt="temperature"
GET /temp               -> 2.05 Content  23.5
GET /humidity           -> 4.04 Not Found  -
```

Analisar

O que significa `ct=0` na descrição do recurso? Que vantagem tem a descoberta para a instalação de dezenas de sensores?

Passo 4 — Perdas e retransmissão

Objetivo. Observar uma mensagem confirmável a ser retransmitida com recuo exponencial.



Como ler a figura

À esquerda, o que o programa faz: envia uma mensagem confirmável, não recebe ACK, espera, retransmite e acaba por receber a confirmação. À direita, a alternativa não confirmável, que não espera por resposta.

Fundamento teórico

Numa mensagem confirmável, o emissor guarda a mensagem e inicia um temporizador. Se o ACK não chegar, retransmite com o mesmo identificador e duplica o tempo de espera. A norma usa um valor inicial de cerca de 2 segundos, com um fator aleatório, e até 4 retransmissões. O programa usa 0,5 segundos, para a demonstração ser rápida. Para o servidor, o mesmo identificador permite reconhecer duplicados.

A analisar neste passo: o tempo de espera em cada tentativa.

Comando a executar

```
python3 coap_lab.py retransmit
```

Resultado esperado (os valores podem variar)

```
server drops the first two requests
  attempt 1: no ACK within 0.5 s, retransmitting
  attempt 2: no ACK within 1.0 s, retransmitting
answered after 2 retransmission(s); payload b'23.5'
```

Analisar

Altere `drop_first` para 5 e preveja o resultado. Quando é que o cliente desiste? Em que aplicações de IoT usaria uma mensagem não confirmável?

Passo 5 — Limpeza

Não ficou nada instalado: basta apagar o ficheiro `coap_lab.py`.

Questões de consolidação

1. Que campos do cabeçalho CoAP permitem associar uma resposta ao pedido correspondente?
2. Porque é que o CoAP não necessita de estabelecer ligação?
3. Como descobre um cliente os recursos de um servidor sem os conhecer?
4. Qual é a consequência de repetir o mesmo identificador de mensagem numa retransmissão?

Síntese

- O **cabeçalho base** do CoAP ocupa 4 octetos e é binário, ao contrário do HTTP, que é textual.
- Sem TCP, não há **ligação** nem três trocas iniciais.
- A **descoberta** é feita em `.well-known/core`.
- A **fiabilidade** é tratada na aplicação: confirmação, retransmissão e recuo exponencial.